

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

## **Лабораторна робота № 11**

з дисципліни «Алгоритмізація та програмування»

на тему: «Розробка десктоп-застосунків в середовищі QtCreator»

XAI.305.

Виконав(ла): здобувач 1 курсу, групи 319  
напряму підготовки (спеціальності) G7  
Автоматизація, комп'ютерно-інтегровані  
технології та робототехніка

Павло Сафонов

*(ім'я і прізвище)*

Прийняла: доц. каф.301, канд. техн. наук,

Олена ГАВРИЛЕНКО

## МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ роботи з низькорівневими рядками на C++ і документацію до класу string, а також алгоритми пошуку в рядку, а також реалізувати обробку рядків на C++ в середовищі QtCreator.

## ПОСТАНОВКА ЗАДАЧІ

### Завдання 1.

А. Вивчити по документації метод стандартного класу string відповідно до варіанту (див. табл.1).

В. Визначити функцію, що виконує ті ж дії, що і вивчений метод класу string. Вихідний рядок передати першим параметром (масив символів). Для реалізації методу не використовувати функції обробки рядків зі стандартних бібліотек.

С. Викликати свій метод і метод string аналогічно прикладам коду, наведеними в дод.А. \*Перед викликом ввести з консолі один рядок і зберегти в масиві символів і змінній типу string.

### Завдання 2.

А.Описати функцію, що обробляє рядок відповідно до завдання з табл.2.

Для

реалізації можна використовувати функції обробки рядків зі стандартних бібліотек

В.Описати функцію, яка перевіряє, чи задовольняє рядок умовам завдання.

С.\* Створити вихідний текстовий файл, що містить не менше 10 різних рядків.

Д.Використовуючи функції 2.А і 2.В, обробити рядок / \* текстовий файл рядок за рядком. Додаткові дані ввести з консолі.

Е. Отриманий результат записати у вихідний файл.

### Завдання 3.

Завдання 1-2 реалізувати окремими функціями без параметрів, у функції main() організувати меню для багаторазового виконання завдань.

Структурувати проєкт програми: винести заголовки і реалізацію функцій в окремі .h та .cpp файли.

Завдання 4. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок

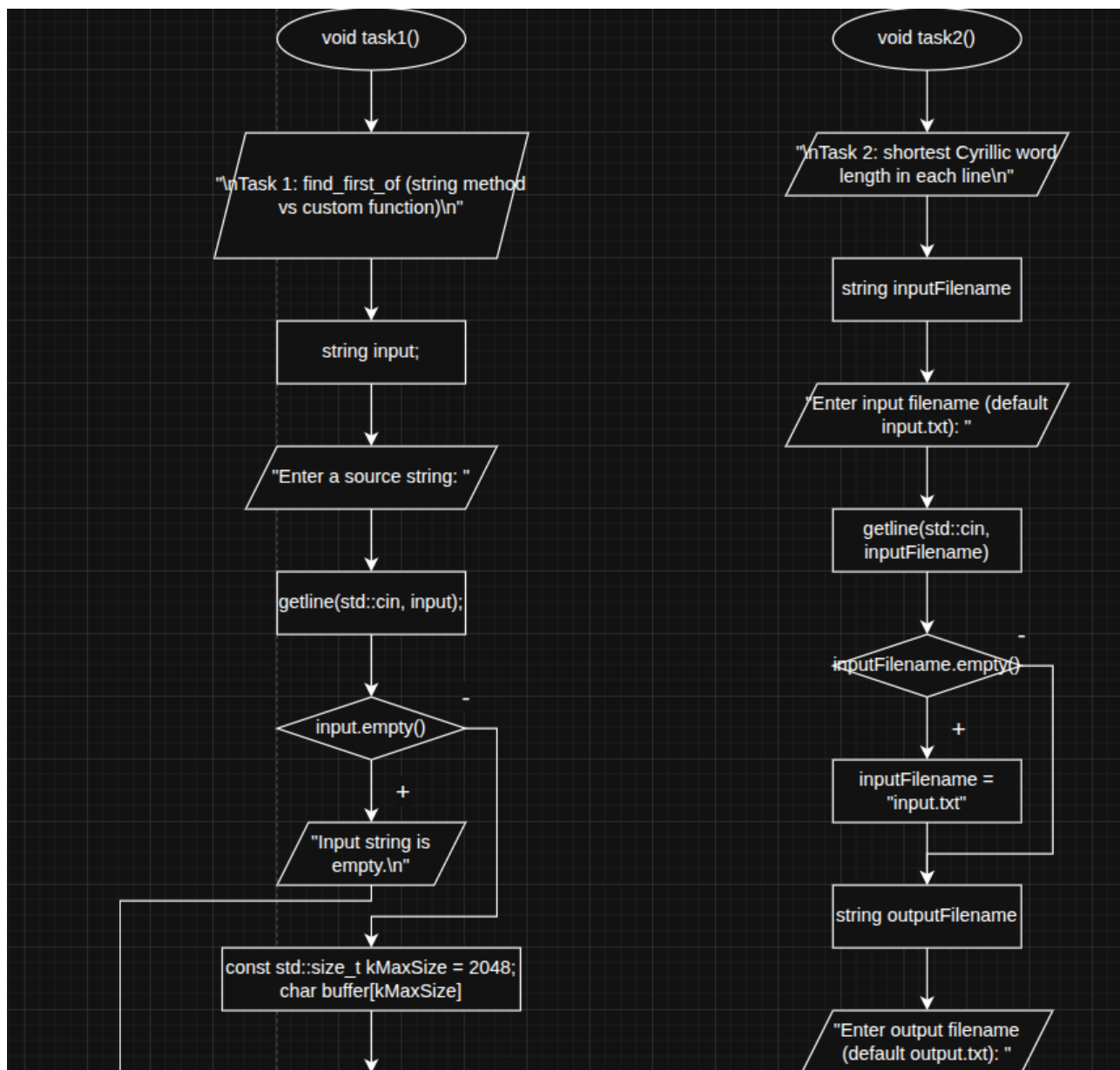
### Завдання 34

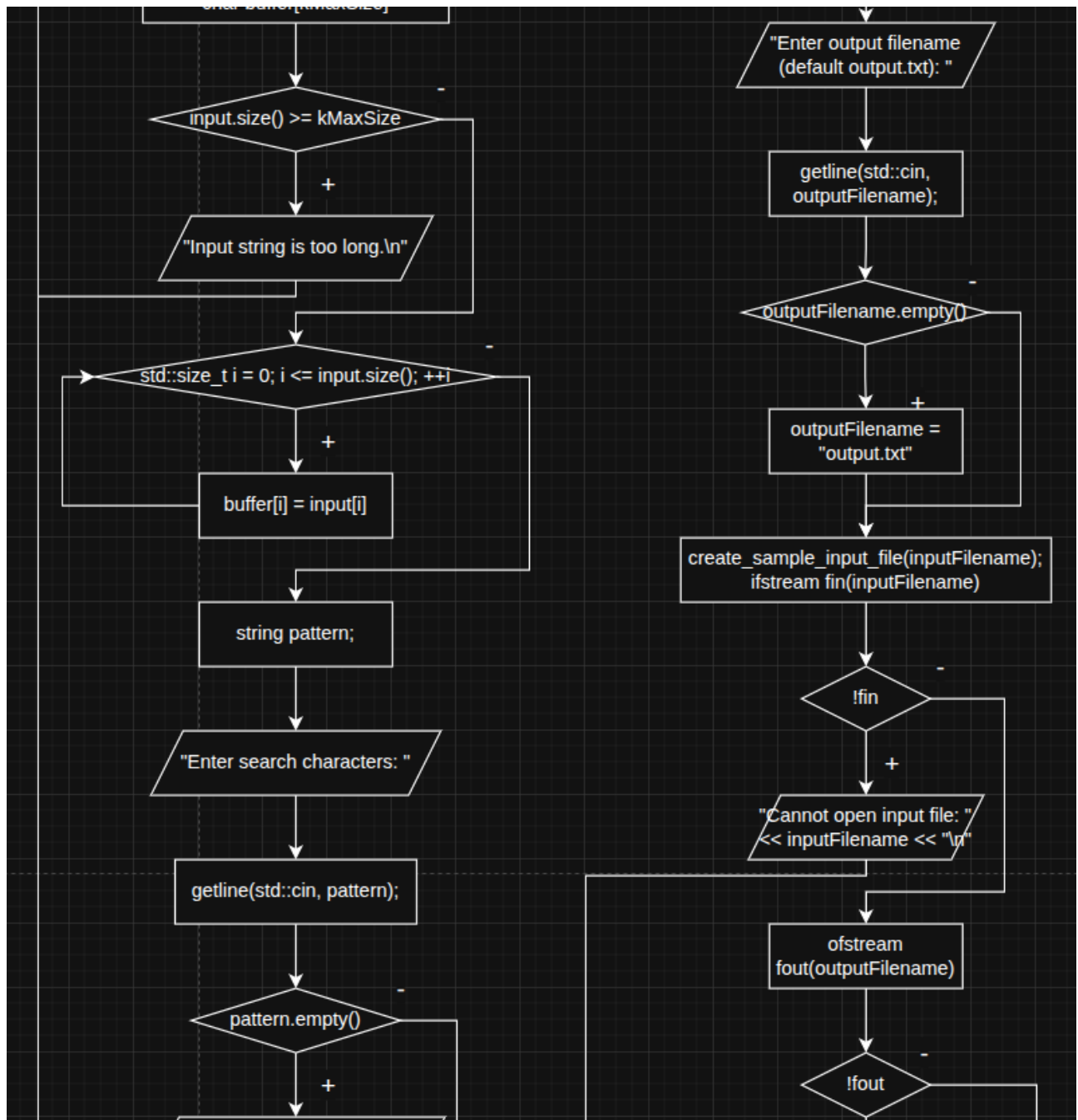
|     |                                                                  |
|-----|------------------------------------------------------------------|
| 34. | size_t find_first_of(const char* s, size_t pos, size_t n) const; |
|-----|------------------------------------------------------------------|

String 45

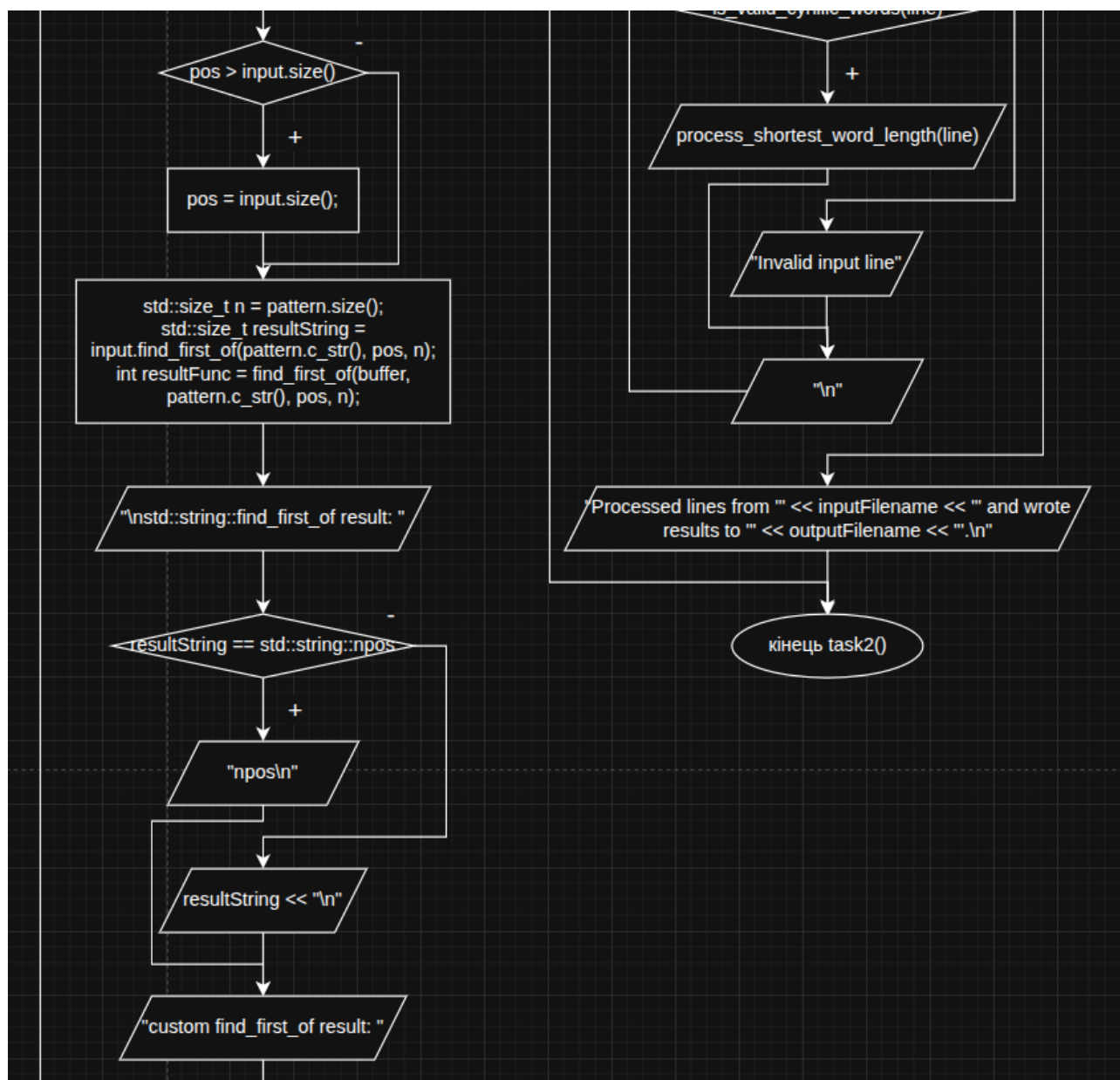
|                  |                                                                                                                               |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>String45.</b> | Дано рядок, що складається з кириличних слів, розділених пробілами (одним або декількома). Знайти довжину найкоротшого слова. |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|

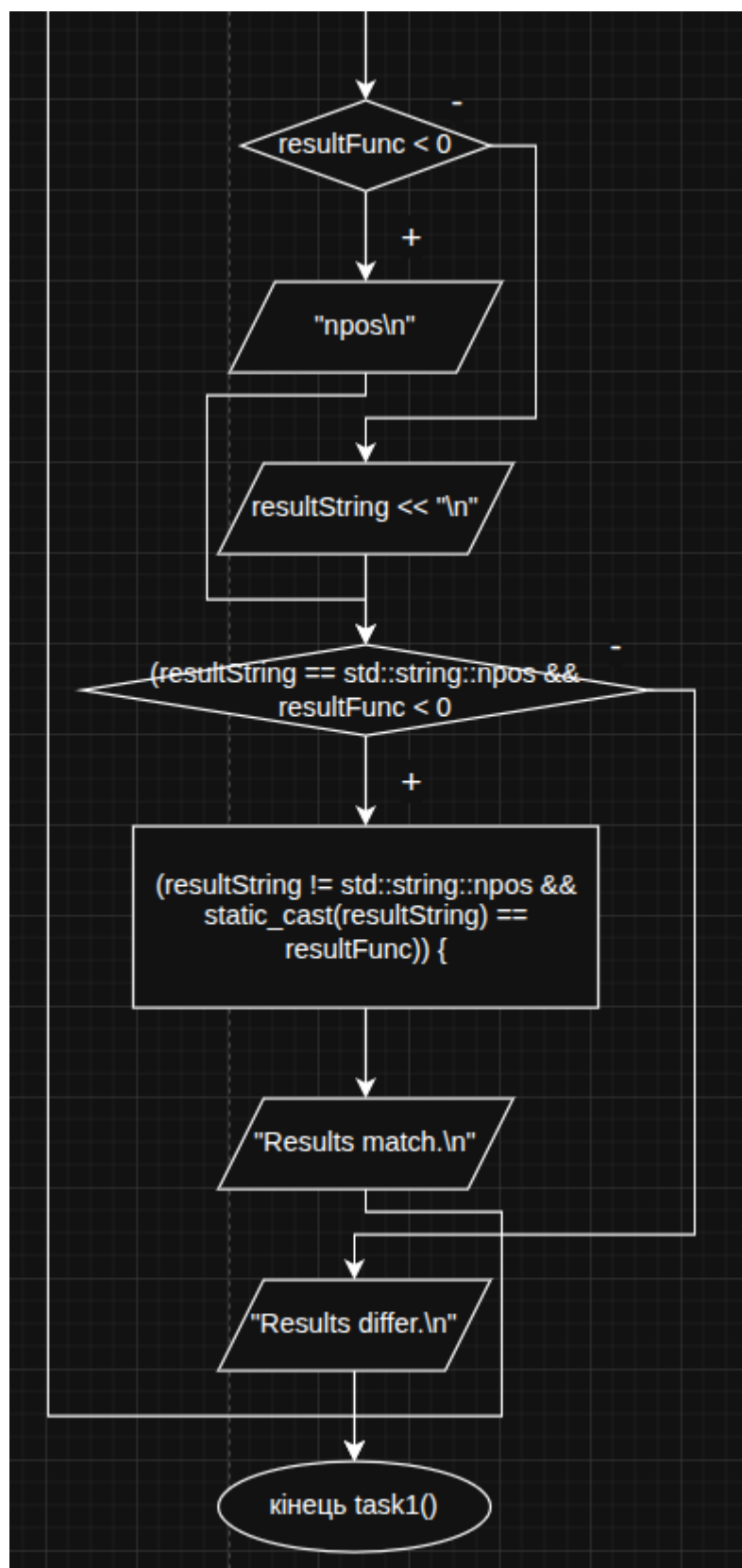
## Блок схеми



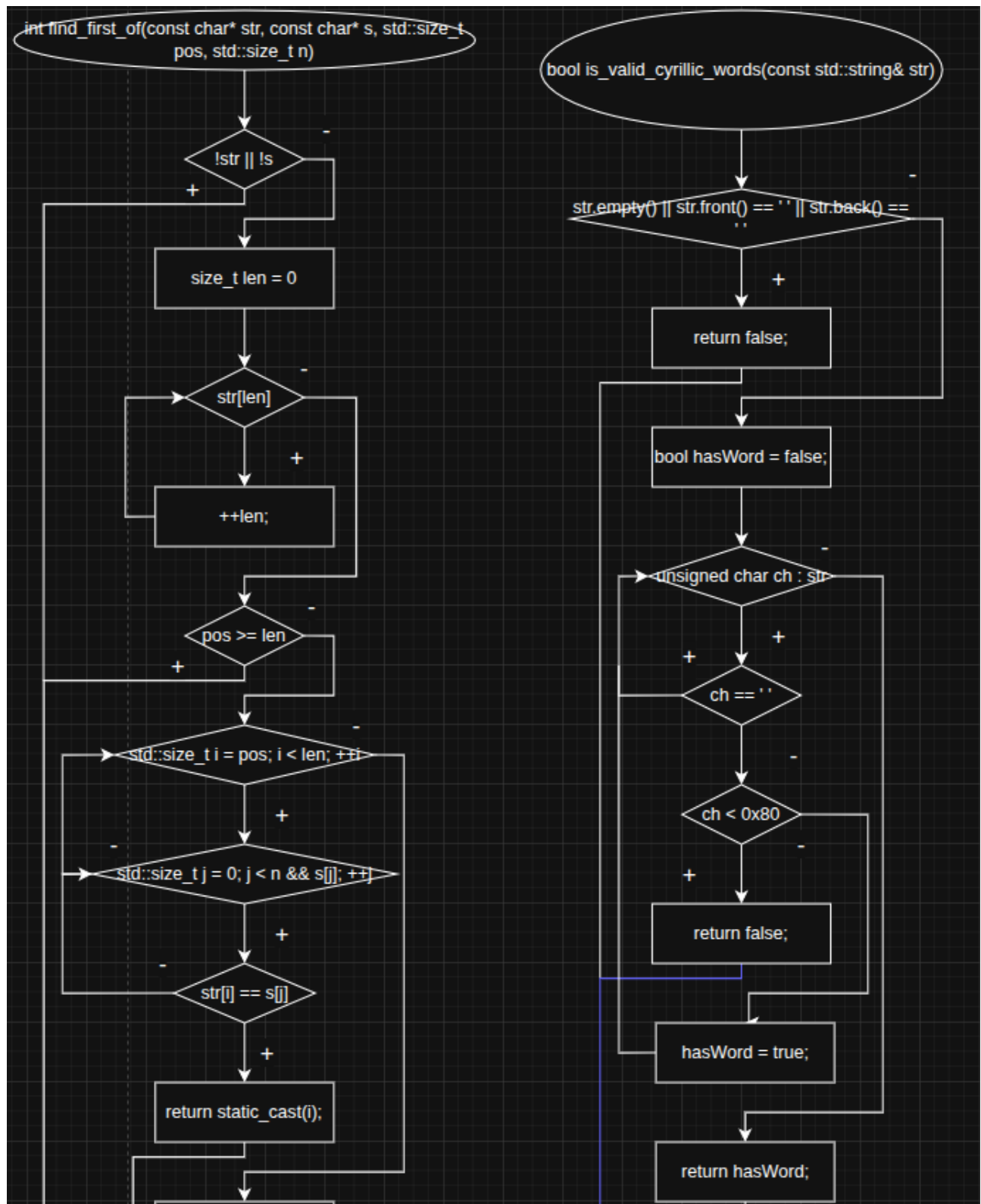


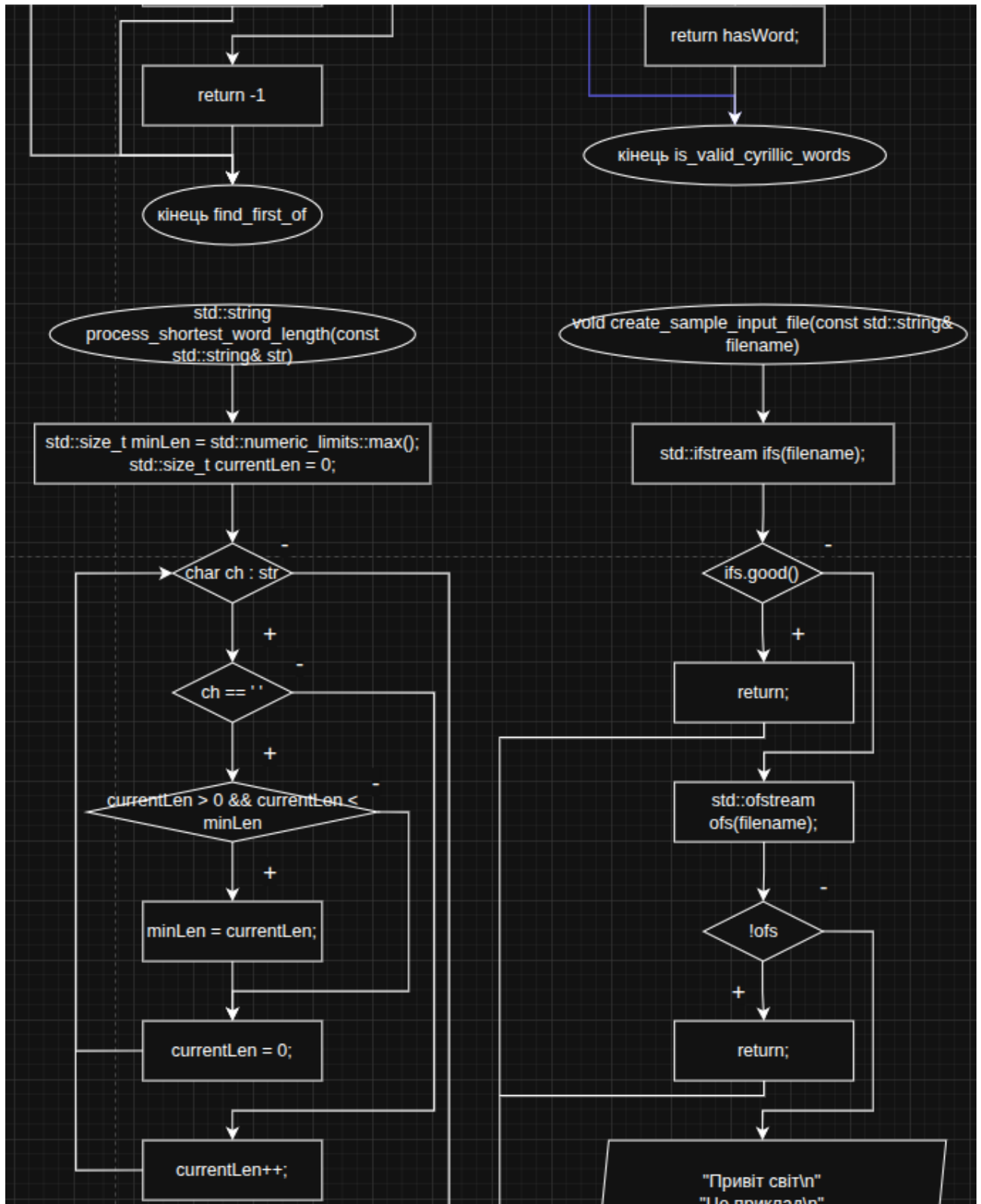


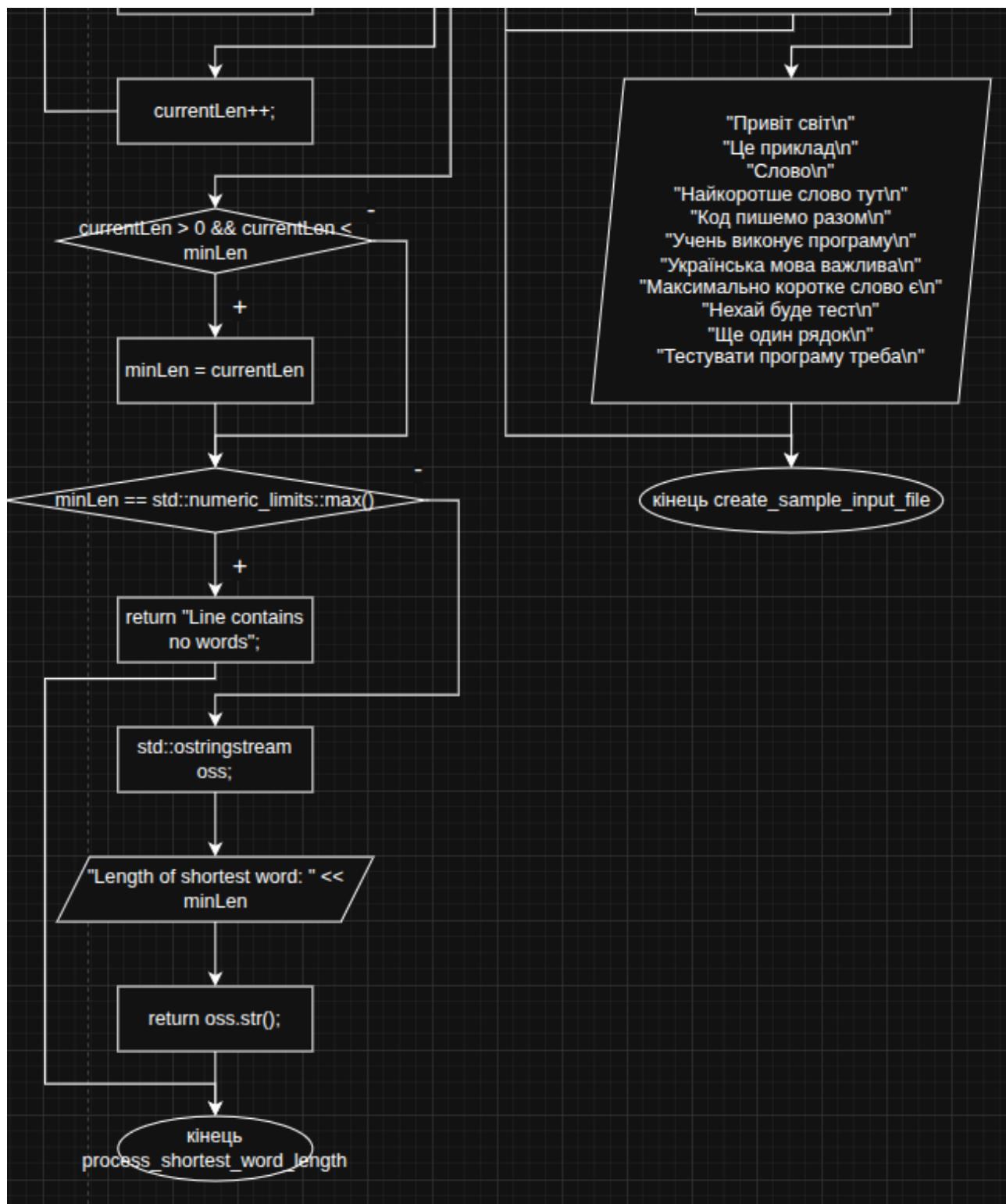












## ДОДАТОК А

```

program.cpp
// Головна точка входу для консольного застосунку.
// Ця програма використовує просте меню для виклику task1 або task2.
#include <iostream>
#include <limits>
#include "tasks.h"

int main()
{
    // Неодноразово показувати користувачеві меню, доки він не вирішить
    вийти.
    while (true) {
        std::cout << "\nMenu:\n";
        std::cout << "1 - Task 1: find_first_of\n";
        std::cout << "2 - Task 2: shortest Cyrillic word length\n";
        std::cout << "0 - Exit\n";
        std::cout << "Enter choice: ";

        int choice = -1;
        if (!(std::cin >> choice)) {
            // Обробляємо нецілочисельний вхід, очищуючи потік та ігноруючи
            решту рядка.
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
                '\n');

            std::cout << "Invalid input. Please enter a number.\n";
            continue;
        }

        // Очищаємо символ нового рядка зліва, зчитуючи числовий вибір.
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

        switch (choice) {
            case 1:
                task1();
                break;
            case 2:
                task2();
                break;
            case 0:
                // Exit the program cleanly.
                std::cout << "Exiting.\n";
                return 0;
            default:
                // Handle any invalid menu option.
                std::cout << "Please choose 0, 1, or 2.\n";
                break;
        }
    }
}

```

```

tasks.cpp
#include "tasks.h"

#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>

// Користувачка функція, що імітує std::string::find_first_of для рядків у
// стилі C.
int find_first_of(const char* str, const char* s, std::size_t pos,
std::size_t n)
{
    if (!str || !s) return -1;

    std::size_t len = 0;
    while (str[len]) {
        ++len;
    }
    if (pos >= len) {
        // Якщо початкова позиція знаходиться за кінцем, нічого не
        // знайдено.
        return -1;
    }

    for (std::size_t i = pos; i < len; ++i) {
        for (std::size_t j = 0; j < n && s[j]; ++j) {
            if (str[i] == s[j]) {
                return static_cast<int>(i);
            }
        }
    }
    return -1;
}

// Обчислюємо довжину найкоротшого слова в рядку, розділеному пробілами.
std::string process_shortest_word_length(const std::string& str)
{
    std::size_t minLen = std::numeric_limits<std::size_t>::max();
    std::size_t currentLen = 0;

    for (char ch : str) {
        if (ch == ' ') {
            // Кінець слова: оновити мінімальну довжину, якщо потрібно.
            if (currentLen > 0 && currentLen < minLen) {
                minLen = currentLen;
            }
            currentLen = 0;
        } else {
            currentLen++;
        }
    }
}

```

```

    }

    if (currentLen > 0 && currentLen < minLen) {
        minLen = currentLen;
    }

    if (minLen == std::numeric_limits<std::size_t>::max()) {
        return "Line contains no words";
    }

    std::ostringstream oss;
    oss << "Length of shortest word: " << minLen;
    return oss.str();
}

// Перевірити, чи рядок містить лише кириличні слова, розділені пробілами.
bool is_valid_cyrillic_words(const std::string& str)
{
    if (str.empty()) {
        return false;
    }
    if (str.front() == ' ' || str.back() == ' ') {
        // Відхилити початкові або кінцеві пробіли відповідно до умов
завдання.
        return false;
    }

    bool hasWord = false;

    for (unsigned char ch : str) {
        if (ch == ' ') {
            continue;
        }
        if (ch < 0x80) {
            // Зустрівся некириличний ASCII-символ.
            return false;
        }
        hasWord = true;
    }

    return hasWord;
}

// Створити вхідний файл зразка за замовчуванням з кириличними рядками,
якщо файл не існує.
void create_sample_input_file(const std::string& filename)
{
    std::ifstream ifs(filename);
    if (ifs.good()) {
        return;
    }

```

```

std::ofstream ofs(filename);
if (!ofs) {
    return;
}

ofs << "Привіт світ\n";
ofs << "Це приклад\n";
ofs << "Слово\n";
ofs << "Найкоротше слово тут\n";
ofs << "Код пишемо разом\n";
ofs << "Учень виконує програму\n";
ofs << "Українська мова важлива\n";
ofs << "Максимально коротке слово є\n";
ofs << "Нехай буде тест\n";
ofs << "Ще один рядок\n";
ofs << "Тестувати програму треба\n";
}

// Завдання 1: порівняння std::string::find_first_of з власною реалізацією.
void task1()
{
    std::cout << "\nTask 1: find_first_of (string method vs custom
function)\n";
    std::string input;
    std::cout << "Enter a source string: ";
    std::getline(std::cin, input);
    if (input.empty()) {
        std::cout << "Input string is empty.\n";
        return;
    }

    const std::size_t kMaxSize = 2048;
    char buffer[kMaxSize];
    if (input.size() >= kMaxSize) {
        std::cout << "Input string is too long.\n";
        return;
    }

    // Копіювання вхідних даних у масив символів у стилі C з нульовим
    завершенням.
    for (std::size_t i = 0; i <= input.size(); ++i) {
        buffer[i] = input[i];
    }

    std::string pattern;
    std::cout << "Enter search characters: ";
    std::getline(std::cin, pattern);
    if (pattern.empty()) {
        std::cout << "Search characters cannot be empty.\n";
        return;
    }

```

```

    }

    std::string positionInput;
    std::cout << "Enter starting position (0.." << input.size() << "): ";
    std::getline(std::cin, positionInput);
    std::size_t pos = 0;
    try {
        pos = std::stoul(positionInput);
    } catch (...) {
        // Якщо парсинг не вдался, за замовчуванням встановлюється позиція
0.        pos = 0;
    }
    if (pos > input.size()) {
        pos = input.size();
    }

    std::size_t n = pattern.size();
    // Порівняти вбудований метод рядків з користувацькою реалізацією.
    std::size_t resultString = input.find_first_of(pattern.c_str(), pos,
n);

    int resultFunc = find_first_of(buffer, pattern.c_str(), pos, n);

    std::cout << "\nstd::string::find_first_of result: ";
    if (resultString == std::string::npos) {
        std::cout << "npos\n";
    } else {
        std::cout << resultString << "\n";
    }

    std::cout << "custom find_first_of result: ";
    if (resultFunc < 0) {
        std::cout << "npos\n";
    } else {
        std::cout << resultFunc << "\n";
    }

    if ((resultString == std::string::npos && resultFunc < 0) ||
        (resultString != std::string::npos &&
static_cast<int>(resultString) == resultFunc)) {
        std::cout << "Results match.\n";
    } else {
        std::cout << "Results differ.\n";
    }
}

// Завдання 2: прочитати рядки з файлу, перевірити кириличні слова та
записати результати.
void task2()
{
    std::cout << "\nTask 2: shortest Cyrillic word length in each line\n";

```



```

std::string inputFilename;
std::cout << "Enter input filename (default input.txt): ";
std::getline(std::cin, inputFilename);
if (inputFilename.empty()) {
    inputFilename = "input.txt";
}

std::string outputFilename;
std::cout << "Enter output filename (default output.txt): ";
std::getline(std::cin, outputFilename);
if (outputFilename.empty()) {
    outputFilename = "output.txt";
}

// Переконалися, що є зразок вхідного файлу, якщо запитуваний не існує.
create_sample_input_file(inputFilename);

std::ifstream fin(inputFilename);
if (!fin) {
    std::cerr << "Cannot open input file: " << inputFilename << "\n";
    return;
}

std::ofstream fout(outputFilename);
if (!fout) {
    std::cerr << "Cannot open output file: " << outputFilename << "\n";
    return;
}

std::string line;
int lineNo = 0;
while (std::getline(fin, line)) {
    ++lineNo;
    if (line.empty()) {
        continue;
    }
    fout << "Line " << lineNo << ": ";
    if (is_valid_cyrillic_words(line)) {
        fout << process_shortest_word_length(line);
    } else {
        fout << "Invalid input line";
    }
    fout << "\n";
}

std::cout << "Processed lines from '" << inputFilename << "' and wrote
results to '" << outputFilename << "'.\n";
}

```

```
#pragma once

// Інтерфейс для реалізації завдань та допоміжних функцій.
#include <string>

// Функції завдання меню.
void task1();
void task2();

// Користувачка реалізація string::find_first_of для рядків у стилі C.
int find_first_of(const char* str, const char* s, std::size_t pos,
std::size_t n);

// Допоміжні засоби обробки рядків для завдання 2.
std::string process_shortest_word_length(const std::string& str);
bool is_valid_cyrillic_words(const std::string& str);

// Створити вхідний файл за замовчуванням, якщо файлу не існує.
void create_sample_input_file(const std::string& filename);
```

## ДОДАТОК Б

```
fenix@Arina:~/learn/vscode$ ./program.exe

Menu:
1 - Task 1: find_first_of
2 - Task 2: shortest Cyrillic word length
0 - Exit
Enter choice: 1

Task 1: find_first_of (string method vs custom function)
Enter a source string: sdfsd
Enter search characters: gg
Enter starting position (0..5): 1

std::string::find_first_of result: npos
custom find_first_of result: npos
Results match.

Menu:
1 - Task 1: find_first_of
2 - Task 2: shortest Cyrillic word length
0 - Exit
Enter choice: 2

Task 2: shortest Cyrillic word length in each line
Enter input filename (default input.txt):
Enter output filename (default output.txt):
Processed lines from 'input.txt' and wrote results to 'output.txt'.

Menu:
1 - Task 1: find_first_of
2 - Task 2: shortest Cyrillic word length
0 - Exit
Enter choice: d
Invalid input. Please enter a number.

Menu:
1 - Task 1: find_first_of
2 - Task 2: shortest Cyrillic word length
0 - Exit
Enter choice: 0
Exiting.
fenix@Arina:~/learn/vscode$
```

ДОДАТОК В

1) «Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань»

1. Для чого у функції `main()` використовується `std::cin.clear()` та `std::cin.ignore()` ?
  - A) Для очищення оперативної пам'яті
  - B) Для відновлення потоку після помилки введення та видалення зайвих символів
  - C) Для відкриття файлу
  - D) Для перевірки кирилиці
2. Яке значення повертає користувацька функція `find_first_of` , якщо символ не знайдено?
  - A) 0
  - B) NULL
  - C) -1
  - D) `npos`
3. Чому у функції `task1()` виконується копіювання `std::string` у масив `char buffer[]` ?
  - A) Для прискорення роботи програми
  - B) Для використання функції, що працює з C-рядками
  - C) Для шифрування рядка
  - D) Для роботи з файлами
4. Що перевіряє функція `is_valid_cyrillic_words()` ?
  - A) Чи містить рядок лише цифри
  - B) Чи містить рядок лише латиницю
  - C) Чи складається рядок із кирилических слів і пробілів
  - D) Чи рядок порожній

5. Яка складність алгоритму користувацької функції `find_first_of` у найгіршому випадку?

- A)  $O(1)$
- B)  $O(\log n)$
- C)  $O(n)$
- D)  $O(n \times m)$

6. Поясніть різницю між `std::string::find_first_of()` та власною реалізацією `find_first_of`. Які переваги та недоліки кожного підходу?

7. Опишіть алгоритм роботи функції `process_shortest_word_length()`.

8. Навіщо у програмі використовується `std::getline()` замість `std::cin >>`?

9. Які помилки введення обробляє програма та як саме?

10. Чому для перевірки кирилиці використовується умова `ch < 0x80`?

- 1) B
- 2) C
- 3) B
- 4) C
- 5) D

Відкриті відповіді

1) Метод `std::string::find_first_of()` є готовою функцією стандартної бібліотеки STL. Вона оптимізована, безпечна та повертає `std::string::npos`, якщо символ не знайдено. Власна реалізація працює з C-рядками (`char*`) та вручну проходить по символах через вкладені цикли. Її перевага — демонстрація принципу пошуку та роботи з масивами символів. Недолік — нижчий рівень безпеки, необхідність контролю нульового символу та більша ймовірність помилок.

2) Функція проходить по кожному символу рядка. Якщо символ не є пробілом — збільшується довжина поточного слова. При зустрічі пробілу перевіряється, чи поточне слово коротше за мінімальне знайдене раніше. Після завершення проходу виконується додаткова перевірка останнього слова. Якщо слова відсутні — повертається повідомлення "Line contains no words".

3) `std::getline()` дозволяє зчитувати весь рядок разом із пробілами. Це необхідно для роботи з реченнями та текстовими рядками. Оператор `>>` зчитує лише до першого пробілу, тому не підходить для задач обробки повних рядків.

4) Програма обробляє:

введення нечислового значення в меню;

порожні рядки;

занадто довгий рядок;

неможливість відкрити файл;

неправильний формат рядків у другому завданні.



Для цього використовуються перевірки `if`, очищення потоку `cin.clear()`, `cin.ignore()`, а також перевірка відкриття файлів через `if (!fin)` і `if (!fout)`.

5) Символи ASCII мають коди від 0 до 127 (0x80). Кириличні символи у UTF-8 мають значення більше 127. Тому перевірка `ch < 0x80` дозволяє виявити латинські символи, цифри та інші ASCII-символи, які не повинні бути у кириличному рядку.

Свернуть ^

Усі тестові відповіді правильні. Відкриті відповіді теж сильні: вони конкретно прив'язані до коду, пояснюють алгоритми та використання STL, тому виглядають природно для захисту роботи.

Оцінювання:

| №  | Оцінка |
|----|--------|
| 1  | 5/5    |
| 2  | 5/5    |
| 3  | 5/5    |
| 4  | 5/5    |
| 5  | 5/5    |
| 6  | 5/5    |
| 7  | 5/5    |
| 8  | 5/5    |
| 9  | 5/5    |
| 10 | 5/5    |

Середній бал: **5.0 / 5**